

RecargaTuLlave

API de Recargas Digitales tuLlave

Documentación técnica

Dato	Valor
Proyecto	RecargaTuLlave
Identificador técnico	TLV-RCG-26
Stack	Java 21 + Spring Boot 3.3.5 + PostgreSQL
Repositorio	https://github.com/SneyderRubio/RecargaTuLlave
Estado	Validado y listo para entrega

1. Resumen de la solución implementada

Se desarrolló una API REST para registrar, consultar y eliminar recargas de tarjetas tuLlave. La aplicación está construida con Java 21 y Spring Boot 3.3.5, utiliza Spring Data JPA para persistencia en PostgreSQL y expone la API en el puerto 8080.

Modelo de datos

Campo	Tipo	Regla
id	Long	Autogenerado
cardNumber	String	16 dígitos numéricos
amount	BigDecimal	2.000 a 200.000
paymentMethod	Enum	PSE, NEQUI, DAVIPLATA, CREDIT_CARD
createdAt	Timestamp	Generado automáticamente

Endpoints

Método	Endpoint	Resultado esperado
POST	/api/v1/recharges	201 Created / 400 Bad Request
GET	/api/v1/getRecharges	200 OK; page, size y cardNumber opcional
DELETE	/api/v1/recharges/{id}	204 No Content / 404 Not Found

Arquitectura

- Controller: recibe solicitudes HTTP, parámetros y DTOs.
- Service: concentra las reglas de negocio y orquesta las operaciones.
- Repository: acceso a datos mediante Spring Data JPA.
- DTOs: separan el contrato HTTP de la entidad de persistencia.
- Mapper: transforma entre DTO y entidad.
- Exception handling: `HandlerError` centraliza respuestas de error.
- Logging: SLF4J en operaciones relevantes sin registrar innecesariamente el número completo de tarjeta.

2. Requisitos cumplidos y decisiones técnicas

Requisito	Implementación / evidencia
Java 21 + Spring Boot 3	Java 21.0.12 y Spring Boot 3.3.5; compilación exitosa.
PostgreSQL	PostgreSQL 16 mediante Docker Compose.
DTO + validación	DTOs y Spring Validation para las entradas de la API.
Errores centralizados	TlvErrorAdvisor con formato consistente.
Capas	Controller / Service / Repository.
Logs	SLF4J en operaciones del servicio.
Docker	Dockerfile multi-stage + docker-compose.yml.
Paginación/filtro	Pageable y filtro opcional por cardNumber.
Swagger	Springdoc OpenAPI / Swagger UI.
Pruebas	JUnit 5 + Mockito; 5 tests exitosos.
Postman	Colección TLV-RCG-26 con casos exitosos y de error.

Decisiones que conviene explicar en la sustentación

- **BigDecimal:** evita errores de precisión propios de tipos de punto flotante para montos monetarios.
- **DTOs:** evita exponer directamente la entidad JPA y permite controlar el contrato de entrada/salida.
- **Enum:** restringe paymentMethod a los medios definidos por el ejercicio.
- **Pageable:** delega la paginación al mecanismo estándar de Spring Data.
- **Docker multi-stage:** separa la etapa de compilación de la imagen final de ejecución.
- **Healthcheck:** permite coordinar el arranque de PostgreSQL y la API en Compose.

3. Procedimiento de ejecución después del clone

3.1. Clonar y entrar al proyecto

```
git clone https://github.com/SneyderRubio/RecargaTuLlave.git cd
RecargaTuLlave
```

```
git status git branch -a
git checkout main git pull origin main
```

Resultado esperado

```
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

3.2. Verificar Docker

```
docker --version
docker compose version
```

3.3. Levantar aplicación + PostgreSQL (ruta recomendada)

```
docker compose up --build
```

Este comando construye la imagen de la API, crea/levanta PostgreSQL y deja la aplicación disponible en el puerto 8080. En una ejecución validada, la imagen se construyó correctamente, PostgreSQL aceptó conexiones y Spring Boot inició Tomcat en el puerto 8080.

3.4. URLs para la sustentación

```
API:      http://localhost:8080
Swagger:  http://localhost:8080/swagger-ui/index.html
OpenAPI:  http://localhost:8080/v3/api-docs
```

4. Pruebas funcionales con Postman

Importar la colección

En Postman:

1. Importar: `postman/TLV-RCG-26.postman_collection.json`
2. Abrir la colección TLV-RCG-26.
3. Usar `baseUrl = http://localhost:8080` si la colección solicita la variable.

Flujo recomendado para demostrar la API

Paso	Petición	Qué demostrar
1	POST <code>/api/v1/recharges</code>	201 Created y respuesta con id
2	GET <code>/api/v1/getRecharges?page=0&size=10</code>	200 OK y paginación
3	GET con <code>cardNumber</code>	Filtrado por número de tarjeta
4	DELETE <code>/api/v1/recharges/{id}</code>	204 No Content
5	DELETE con id inexistente	404 Not Found
6	POST con datos inválidos	400 Bad Request y validación

Smoke test automatizado

```
# PowerShell, desde la raíz del proyecto
./scripts/smoke-test.ps1
```

En la validación realizada, el smoke test creó una recarga con 201, consultó las recargas con 200 y eliminó la recarga creada con 204. La respuesta de DELETE incluyó el encabezado X-Message: Recharge deleted successfully.

5. Pruebas automatizadas y ejecución local sin Docker

5.1. Pruebas unitarias

```
.\tools\mvnd\bin\mvn.cmd test
```

Resultado validado: Tests run: 5, Failures: 0, Errors: 0, Skipped: 0 — BUILD SUCCESS.

5.2. Empaquetar el proyecto

```
.\tools\mvnd\bin\mvn.cmd clean package
```

5.3. Verificación de Java

```
java -version
```

Debe reportar Java 21. El proyecto conserva TLV-RCG-26 como artifactId técnico en pom.xml; el nombre del proyecto/documentación es RecargaTuLlave.

6. Comandos de Docker para la sustentación

Ver contenedores

```
docker ps
docker ps -a
```

Ver logs de la API

```
docker logs -f tlv-rcg-26-api
```

Ver logs de PostgreSQL

```
docker logs -f tlv-rcg-26-postgres
```

Detener el entorno

```
docker compose down
```

Reiniciar desde cero incluyendo volumen de PostgreSQL

```
docker compose down -v
docker compose up --build
```

7. Git y flujo de trabajo

Flujo utilizado

- main como rama de integración.
- feature/recharge-api para la funcionalidad principal.
- feature/docker para Docker, documentación y scripts.
- chore/update-readme-tools para la actualización final de documentación y herramientas Maven.
- Pull Requests #1, #2 y #3 integrados en main.

Historial final relevante

```
92fd2e2 Merge pull request #3 ... chore/update-readme-tools ca545c0 chore: include Maven tools and update documentation c365046 Merge pull request #2 ... feature/docker
0344e4f Merge pull request #1 ... feature/recharge-api 7b729f4 chore: add project scripts
3139e52 test: add Postman collection
249d550 docs: add project
documentation beed67d feat: add
docker configuration
```

Comandos para demostrar el estado del repositorio

```
git status
git log --oneline --decorate
-8 git branch -a
git pull origin main
```

Estado validado al cierre: main sincronizada con origin/main y working tree limpio